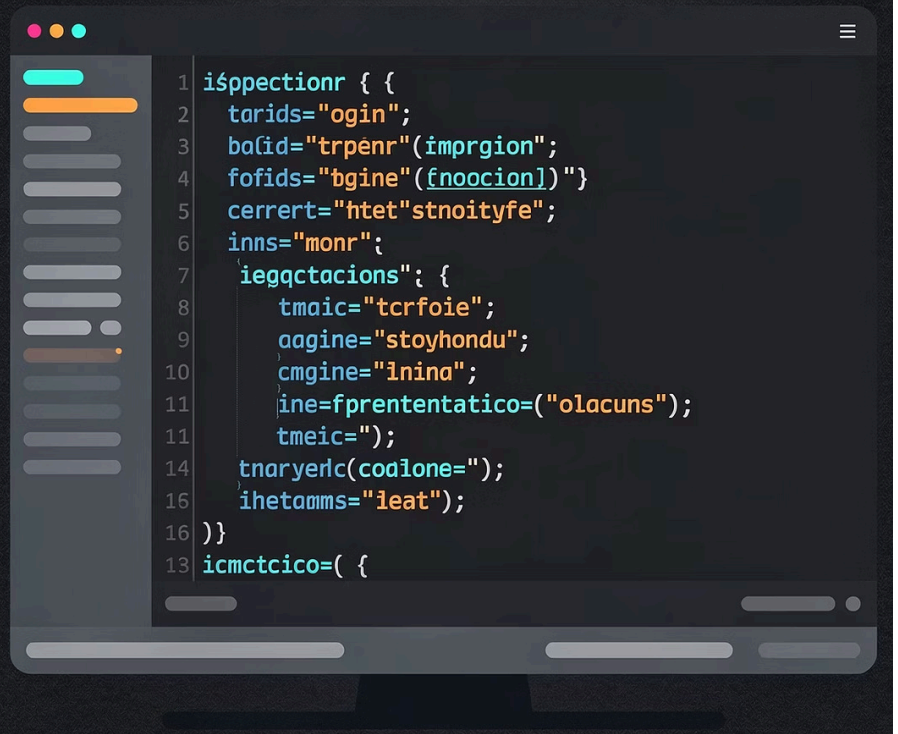


Módulo 3: Control de Flujo y Arreglos de Datos (Arrays)

En este módulo aprenderás a controlar el comportamiento de tus programas mediante sentencias condicionales, ciclos y estructuras de datos como los arreglos. Cubriremos 5 horas de contenido práctico con ejemplos en **Python** y comparaciones con C#.

MÓDULO 3 | 5 HORAS

MsC Manuel Maldonado — DGTIC, UNAM

Agenda del curso

¿Qué veremos hoy?

01

Sentencias Incondicionales

Asignación, lectura, escritura y la historia del temido `goto`.

02

Sentencias Condicionales

Estructuras `if`, `if-else`, `if-elif-else` para tomar decisiones.

03

Ciclos

Repetición con `while`, `do-while` y `for`. Selección múltiple con `match-case`.

04

Arreglos de Datos

Vectores unidimensionales y matrices bidimensionales con ejemplos prácticos.

Módulo 3.1 — Sentencias Incondicionales

La Asignación

La asignación es la operación más fundamental de la programación: permite **guardar un valor en una variable**. En Python y la mayoría de lenguajes modernos se usa el símbolo `=`.

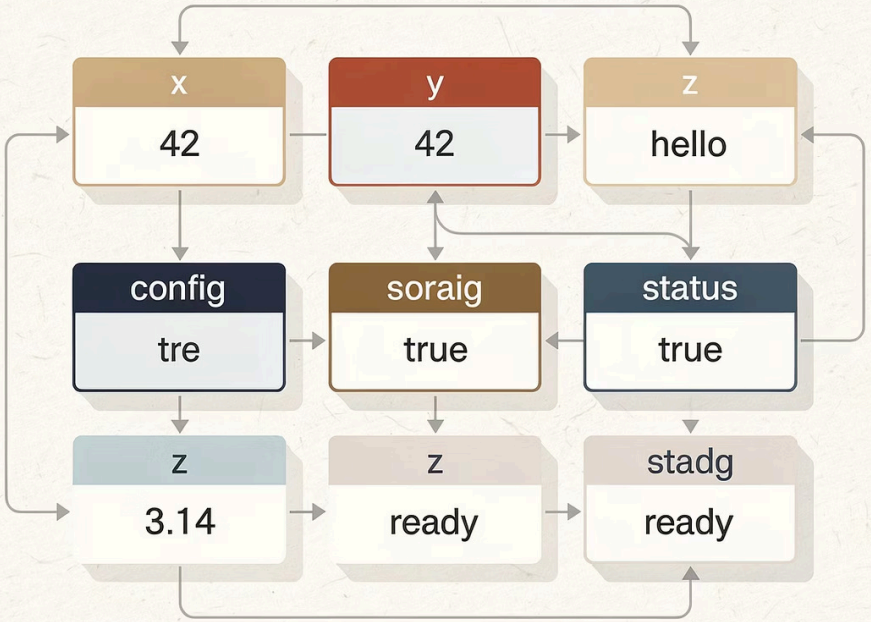
Python (tipado dinámico)

```python
edad = 25
nombre = "Ana"
es_estudiante = True

# Asignación múltiple
x, y, z = 10, 20, 30

# Incremento
contador = 0
contador = contador + 1
```

C# (tipado estático)

```csharp
int edad = 25;
string nombre = "Ana";
bool esEstudiante = true;
```

En C# debes declarar el tipo de dato explícitamente. Python lo infiere automáticamente — ventaja para aprendizaje.



Módulo 3.1.2 — Sentencias Incondicionales

Lectura y Escritura de Datos

📥 Lectura (Entrada)

La función `input()` siempre devuelve un **string**. Si necesitas un número, debes convertirlo explícitamente.

```
nombre = input("¿Cómo te llamas? ")
edad = int(input("¿Cuántos años tienes? "))

print(f"Hola {nombre}, el año que
viene tendrás {edad + 1} años")
```

📤 Escritura (Salida)

La función `print()` muestra información en consola. Los **f-strings** son la forma más moderna y recomendada.

```
print("Hola mundo")
print("El resultado es:", 42)

# f-strings (recomendado)
a, b = 5, 3
print(f"{a} + {b} = {a + b}")
```

💡 **Consejo:** Los f-strings (disponibles desde Python 3.6) son la forma más clara y eficiente de formatear texto. ¡Úsalos siempre que puedas!

Módulo 3.1.4 — Historia de la Programación

El `goto`: Una Reliquia del Pasado

`goto` es una instrucción que salta incondicionalmente a otra parte del código. Fue popular en lenguajes como BASIC, Fortran y ensamblador. En 1968, **Edsger Dijkstra** publicó el influyente artículo *"Go To Statement Considered Harmful"*, demostrando que su uso excesivo creaba el llamado **"código espagueti"**: imposible de leer y mantener.

```
// En C (aún existe, pero no se usa)
int i = 0;
inicio:
  printf("%d ", i);
  i++;
  if (i < 5) goto inicio;
```

🚫 Python no tiene goto

¡Y eso es una buena noticia! Python promueve el código limpio y estructurado.

✅ Alternativa correcta

Usa ciclos (`for`, `while`) y funciones para estructurar tu código.

🔧 ¿Y si crees que lo necesitas?

Refactoriza tu código. Siempre hay una solución más limpia.

Módulo 3.2 — Sentencias Condicionales

Selección: `if`, `if-else` y `if-elif-else`

Las estructuras condicionales permiten que el programa tome **decisiones** según si una condición es verdadera o falsa. Son la base de cualquier lógica de negocio.

if-else simple

```
edad = 18

if edad >= 18:
    print("Eres mayor de edad")
else:
    print("Eres menor de edad")
```

if-elif-else (múltiples condiciones)

```
nota = 85

if nota >= 90:
 calificacion = "A"
elif nota >= 80:
 calificacion = "B"
elif nota >= 70:
 calificacion = "C"
else:
 calificacion = "F"

print(f"Tu calificación: {calificacion}")
```

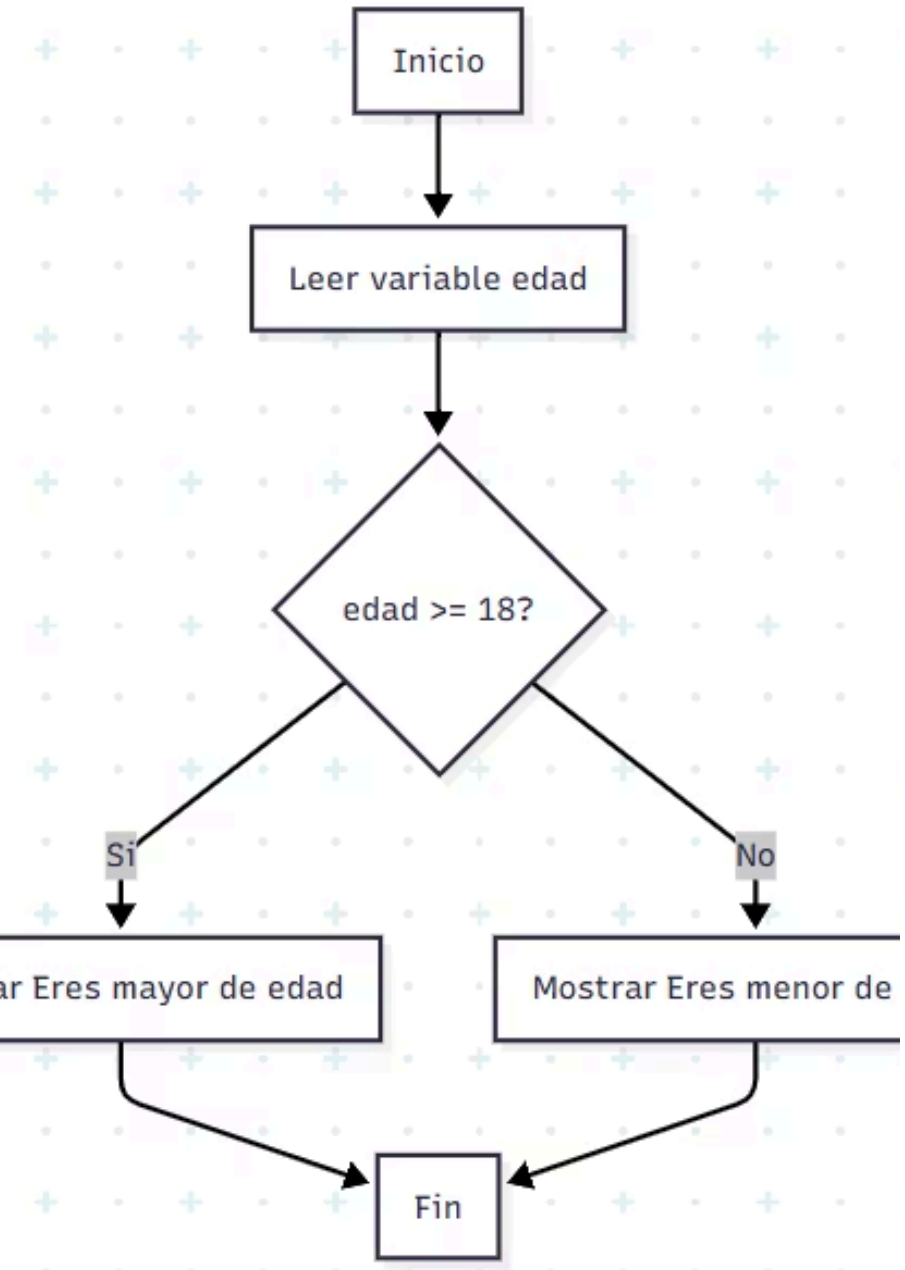


Diagrama de Flujo — if-else

¿Cómo funciona visualmente?

Este diagrama ilustra el flujo de ejecución de una estructura `if-else`. El programa evalúa la condición y toma uno de dos caminos posibles.

📌 El rombo representa una **decisión**. Siempre tiene exactamente dos salidas: **Sí** y **No**. Los rectángulos son acciones o instrucciones.

Módulo 3.2.2 — Ciclos

Ciclo while: Repetir mientras...

El ciclo `while` ejecuta un bloque de código **mientras la condición sea verdadera**. Es ideal cuando no sabes de antemano cuántas veces se repetirá el ciclo. ⚠️ Siempre actualiza la variable de control para evitar ciclos infinitos.

while en Python

```python
contador = 1
while contador <= 5:
    print(f"Vuelta {contador}")
    contador += 1
# Salida: Vuelta 1, Vuelta 2... Vuelta 5
```

Simulación de do-while en Python

```python
i = 1
while True:
    print(f"Vuelta {i}")
    i += 1
    if i > 5:
        break  # Salir del ciclo
```

do-while en C# (ejecuta al menos una vez)

```csharp
int i = 1;
do {
    Console.WriteLine($"Vuelta {i}");
    i++;
} while (i <= 5);
```

🔑 **Diferencia clave:** `do-while` evalúa la condición *después* de ejecutar el bloque, garantizando al menos una ejecución. Python no lo tiene nativamente, pero se simula con `while True` + `break`.

Módulo 3.2.2 — Ciclos

Ciclo `for`: Iterar con precisión

El ciclo `for` es ideal para recorrer secuencias o ejecutar un número **conocido de iteraciones**. En Python es muy versátil: funciona con rangos, listas, y más.

```
# for con range(inicio, fin, paso)
for i in range(1, 6):       # Del 1 al 5
    print(f"Número {i}")

# for para recorrer listas
frutas = ["manzana", "pera", "uva"]
for fruta in frutas:
    print(f"Me gusta la {fruta}")

# for con enumerate (índice y valor)
for idx, fruta in enumerate(frutas):
    print(f"{idx}: {fruta}")
```

`range(5)`

0, 1, 2, 3, 4 — Desde 0 hasta 4

`range(1, 6)`

1, 2, 3, 4, 5 — Desde 1 hasta 5

`range(0, 10, 2)`

0, 2, 4, 6, 8 — De 2 en 2

Comparativa de Ciclos

¿Cuándo usar cada ciclo?

Tipo	Cuándo usarlo	Python	C# / Java
`while`	Cuando no sabes cuántas iteraciones serán	`while cond:`	`while (cond) { }`
`do-while`	Al menos una iteración garantizada	`while True + break`	`do { } while (cond);`
`for`	Número conocido de iteraciones o recorrer colecciones	`for i in range(10):`	`for (int i=0; i<10; i++) { }`

Módulo 3.2.3 — Selección Múltiple

Switch / match-case

Cuando tienes muchas opciones posibles, una cadena de `if-elif-else` puede volverse difícil de leer. La selección múltiple (`switch` en C#, `match-case` en Python 3.10+) ofrece una **sintaxis más limpia y expresiva**.

Python 3.10+ — match-case

```python
opcion = 2

match opcion:
    case 1:
        print("Opción 1")
    case 2:
        print("Opción 2")
    case 3:
        print("Opción 3")
    case _:   # default
        print("Opción no válida")
```

C# — switch expression (moderno)

```csharp
string resultado = opcion switch {
    1 => "Opción 1",
    2 => "Opción 2",
    _ => "No válida"
};
```

Python < 3.10 (alternativa)

```python
opciones = {
 1: "Opción 1",
 2: "Opción 2",
 3: "Opción 3"
}
print(opciones.get(opcion, "No válida"))
```

Módulo 4 — Arreglos de Datos

¿Qué es un Arreglo?

Un **arreglo** (array) es una estructura que almacena una **colección de elementos** bajo un mismo nombre, accesibles mediante un índice numérico. En Python se implementan con listas (`[ ]`). Los índices siempre comienzan en **0**.

Tamaño

Fijo en C# y Java. **Dinámico** en Python (puedes agregar o quitar elementos).

Acceso directo

Puedes acceder a cualquier elemento en tiempo constante usando su índice: `edades[2]`

Índice base 0

El primer elemento es siempre el índice `0`. El último es `len(arreglo) - 1`.

Módulo 4.2 — Arreglos Unidimensionales

Vectores en Python

Operaciones básicas

```python
numeros = [10, 20, 30, 40, 50]

# Acceso
print(numeros[0])  # 10
print(numeros[-1])  # 50 (último)

# Modificación
numeros[2] = 99
print(numeros)  # [10, 20, 99, 40, 50]

# Longitud
print(len(numeros))  # 5

# Recorrer con índice
for i in range(len(numeros)):
    print(f"Pos {i}: {numeros[i]}")
```

🧮 Ejemplo práctico: Promedio de calificaciones

```python
calificaciones = [85, 90, 78, 92, 88]
suma = 0

for calif in calificaciones:
    suma += calif

promedio = suma / len(calificaciones)
print(f"El promedio es: {promedio:.2f}")
# Salida: El promedio es: 86.60
```

💡 `:.2f` en el f-string formatea el número con exactamente 2 decimales. ¡Muy útil para mostrar resultados limpios!

Módulo 4.3 — Arreglos Bidimensionales

Matrices: Arrays en 2 Dimensiones

Una **matriz** es un arreglo de arreglos — como una tabla con **filas y columnas**. Se accede con dos índices: `[fila][columna]`. En Python se implementan con listas anidadas.

Creación y acceso

```python
matriz = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

print(matriz[0][0])  # 1
print(matriz[1][2])  # 6

# Modificar
matriz[2][1] = 99

# Recorrer con doble for
for fila in range(len(matriz)):
    for col in range(len(matriz[fila])):
        print(f"[{fila}][{col}]={matriz[fila][col]}")
```

Visualización conceptual

	Col 0	Col 1	Col 2
Fila 0	1	2	3
Fila 1	4	5	6
Fila 2	7	8	9

`matriz[1][2]` → **6** (fila 1, columna 2)

En C# se usa `int[,] m = new int[3,3]` y se accede con `m[1,2]`.

Módulo 4.3 — Ejemplo Práctico

Suma de Matrices 2×2

Sumar dos matrices consiste en sumar elemento a elemento: cada posición `[i][j]` de la matriz resultado es la suma de las posiciones correspondientes de `A` y `B`.

```
A = [[1, 2],
     [3, 4]]

B = [[5, 6],
     [7, 8]]

resultado = [[0, 0],
             [0, 0]]

for i in range(2):
    for j in range(2):
        resultado[i][j] = A[i][j] + B[i][j]

for fila in resultado:
    print(fila)
# Salida:
# [6, 8]
# [10, 12]
```

Actividades Prácticas

¡Hora de practicar!

La mejor manera de aprender a programar es **escribiendo código**. A continuación encontrarás los ejercicios correspondientes a cada hora del módulo.

Actividades — Horas 1, 2 y 3

Ejercicios: Sentencias y Ciclos

1	**Saludo personalizado** Lee nombre, edad y ciudad con `input()` y muestra un saludo con f-strings. Practica asignación y escritura.
2	**Debate: ¿Por qué goto es dañino?** Discusión grupal con ejemplos históricos. Investiga el artículo de Dijkstra de 1968.
3	**Calculadora de IMC** Calcula el Índice de Masa Corporal usando `if-elif-else`. Clasifica: bajo peso, normal, sobrepeso, obesidad.
4	**Juego de adivinanza** Usa `while` para que el usuario adivine un número secreto. Da pistas "mayor" o "menor" en cada intento.
5	**Menú interactivo** Crea un menú con opciones usando `match-case` (Python 3.10+) o un diccionario como alternativa.
6	**Tabla de multiplicar** Usa `for` para imprimir la tabla de multiplicar del 1 al 10 para cualquier número ingresado por el usuario.

Actividades — Horas 4 y 5

Ejercicios: Arreglos y Matrices

Estadísticas de un arreglo

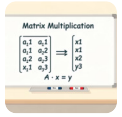
Almacena 5 números ingresados por el usuario en una lista. Calcula y muestra la **suma, promedio, máximo y mínimo** sin usar funciones predefinidas como `max()` o `min()`.

Invertir un arreglo

Invierte el orden de los elementos de una lista **sin usar** `reverse()` **ni slicing**. Usa índices y un algoritmo propio de intercambio.

Matriz identidad 4×4

Crea una matriz de 4×4 con **unos en la diagonal principal** y ceros en el resto. Imprímela de forma visual en la consola.

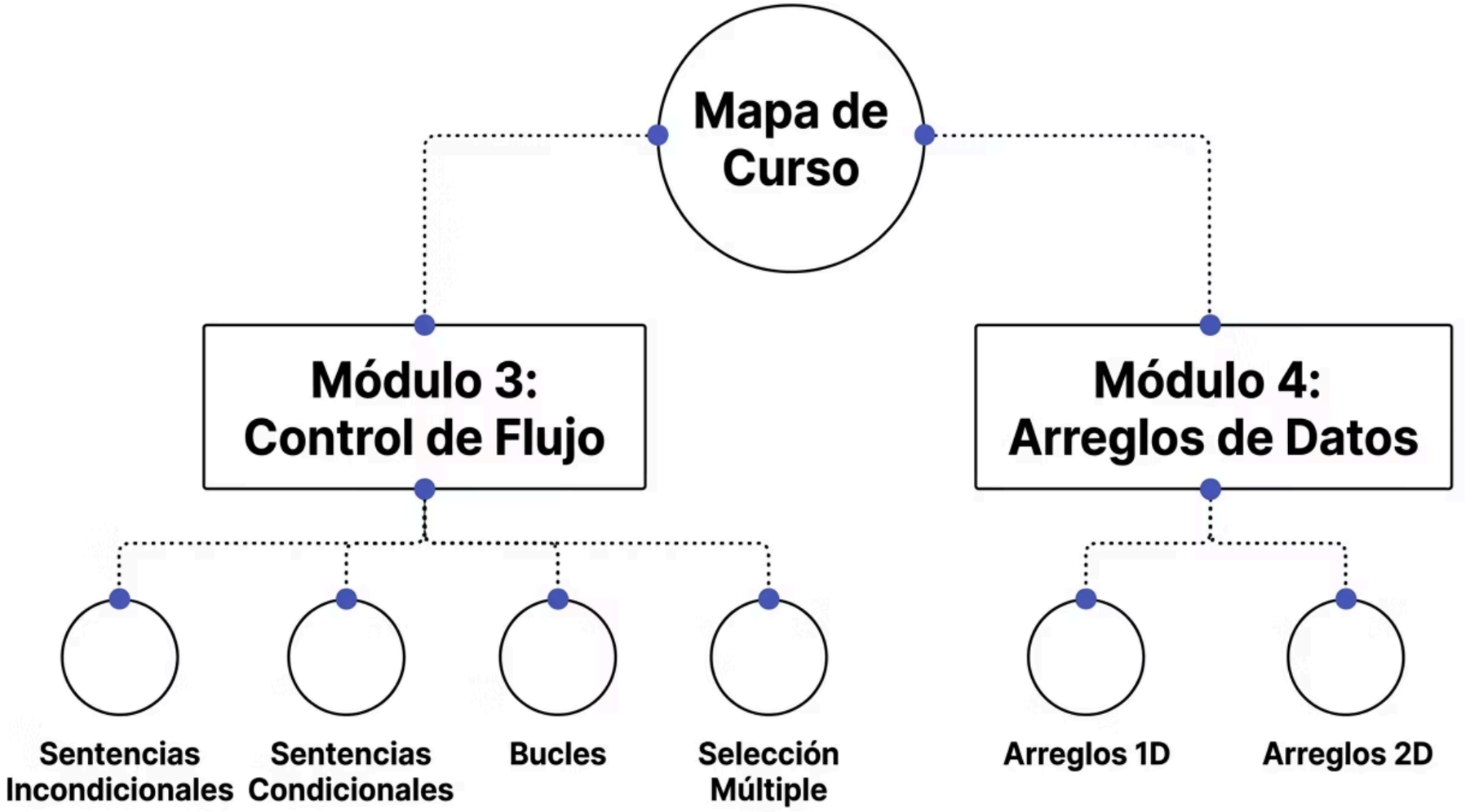


Multiplicación de matrices 2×2

Implementa el **producto matricial** de dos matrices 2×2. Recuerda: `C[i][j] = Σ A[i][k] * B[k][j]`. ¡El más desafiante del módulo!

Resumen Visual

Mapa del Módulo 3 y 4



Estos dos módulos forman la base del **pensamiento algorítmico estructurado**. Dominarlos te dará las herramientas para resolver la gran mayoría de problemas de programación básica e intermedia.

Cierre del Módulo

Puntos Clave para Recordar

La asignación, lectura y escritura son los bloques fundamentales

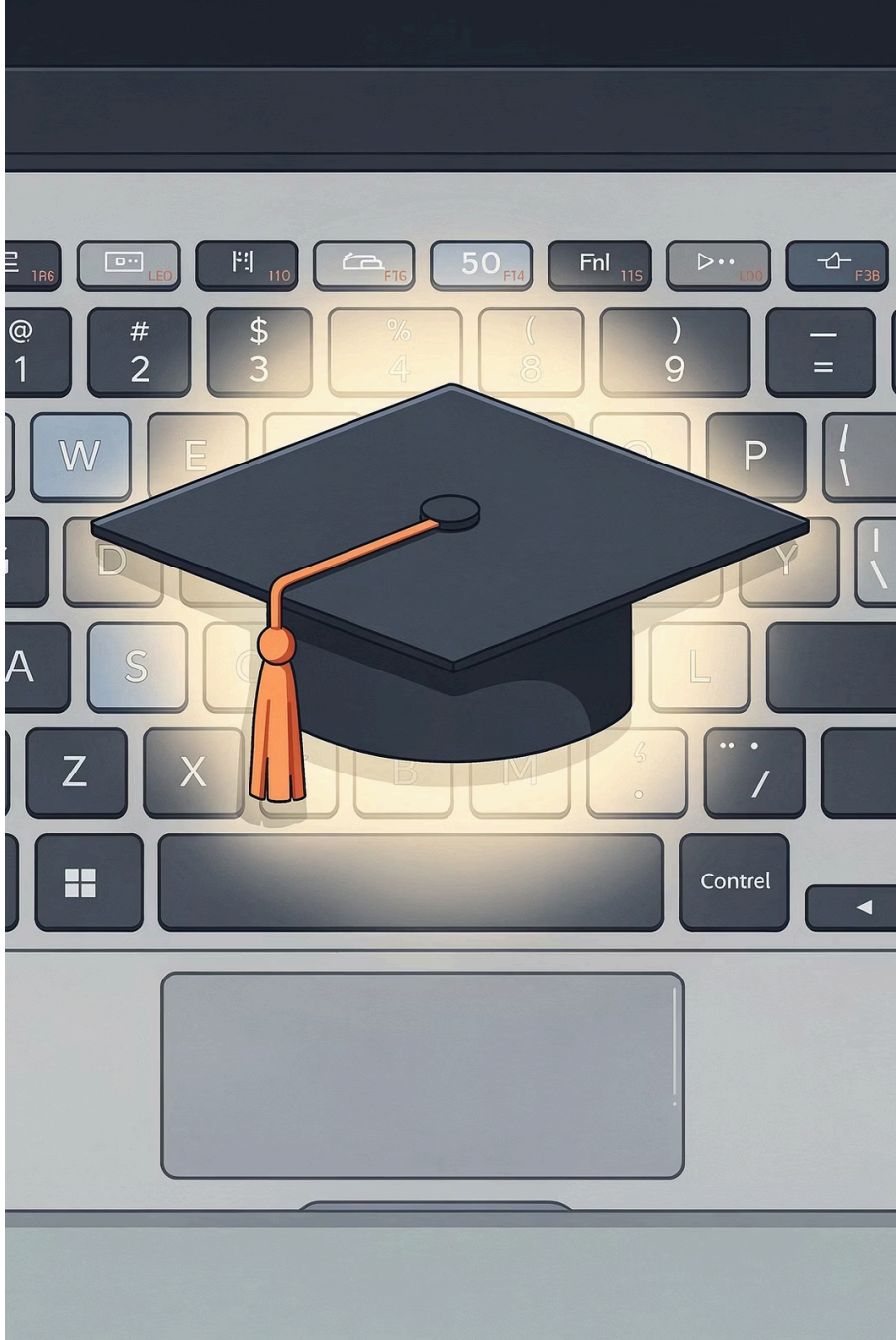
Todo programa se construye sobre estas tres operaciones básicas.

Los condicionales y ciclos dan vida a la lógica

Con `if`, `while` y `for` puedes resolver prácticamente cualquier problema algorítmico.

Los arreglos organizan datos de forma eficiente

Vectores para colecciones simples, matrices para datos tabulares o en cuadrícula.

Python es más limpio, C# más explícito

Ambos son válidos. Python te enseña los conceptos; C# te prepara para entornos empresariales.